МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
Военный факультет

ОТЧЕТ
по контрольной работе № 3

Курсант 3 курса К.А. Вруновский
дневного отделения
“Прикладная криптография”

Преподаватель М.И. Давидовская

Минск, 2026

Задача 1

1. **Что такое (), {}, и $() в Bash?**

   () — **subshell**. Команды внутри () выполняются в **отдельном процессе** (child shell). Изменения внутри subshell (переменные, текущая директория, traps и т.д.) **не влияют** на родительскую оболочку. **{}** — **command grouping**. Команды внутри { } выполняются **в текущей оболочке**. Это просто группировка, как скобки в математике. Важно: после { должен быть пробел, а перед } — ;. **$()** — **command substitution** (подстановка команды). Выполняет команду и **подставляет** её вывод в строку.

   (cd /tmp; pwd) — команды выполняться в отдельной оболочке. Основная оболочка не измениться.
   { cd /tmp; pwd } — команды выполняться в текущей оболочке. По завершению выполнения основная оболчка будет находиться в /tmp.

2. **Разница между source script.sh, bash script.sh и ./script.sh**

   source script.sh: скрипт выполняется **в текущей оболочке**. Все изменения, которые происходят внутри скрипта (переменные, export, cd, функции, alias, изменение PATH, PS1 и т.д.), **остаются** после завершения скрипта. Никакого нового процесса не создаётся.

   bash script.sh и **./script.sh** Скрипт запускается в **новой отдельной оболочке**. После его завершения текущая оболочка **не меняется** — все изменения внутри скрипта пропадают. Переменные окружения передаются только если они были **экспортированы** (export) в родительской оболочке.

   Пример с необходимым source:
   Настройка конфигураций проекта или приложения:

```
#!/bin/bash

# project-env.sh

export PROJECT_ROOT="$(pwd)"

export PYTHONPATH="$PROJECT_ROOT/src"
export DB_HOST="localhost"

export DB_PORT="5432"
```

```
export DB_NAME="myapp"

echo “Config complete.”
```

Пример с плохим использованием source:
Скрипт, который во время своей работы переходит в другие директории.

```
#!/bin/bash

cd ~/

# Тут скрипт выполняет действия в домашней

# директории

...

ls
```

Задача 2

Результат работы:

```
user@user-VirtualBox:~/linux-labs$ #Before:
user@user-VirtualBox:~/linux-labs$ ls -l ./test_dir/
total 0
-rw-rw-r-- 1 user user 0 Apr 29 16:43 file.txt
user@user-VirtualBox:~/linux-labs$ #After:
user@user-VirtualBox:~/linux-labs$ ./rename_by_date.sh ./test_dir/
File renamed.

===Done===
Files renamed: 1
Files skipped: 0
Backup: .rename_backup_1777470226.txt
user@user-VirtualBox:~/linux-labs$ ls -l ./test_dir/
total 0
-rw-rw-r-- 1 user user 0 Apr 29 16:43 file_2026.04.29.txt
user@user-VirtualBox:~/linux-labs$
exit
Script done.
user@user-VirtualBox:~/linux-labs$
```

Исходный код в репозитории.